Design layer

the whole layer, as one document

Contents

1 Retrace 4

1.1 Foundation 4
1.2 System and ownership 4
1.3 Recording and replay 5
1.4 Client delivery 5
1.5 Ingestion and archival 6
1.6 Lustre replay composition 9
1.7 Example application and viewer 11

Glossary 12

ROOT DESIGN

Retrace

1 Retrace

1.1 Foundation

Retrace records application messages, delivers durable sessions and reconstructs them for inspection.

GOAL

Follow a session across capture, storage and replay

A developer can reproduce an application state from a web or Tauri session, search recordings by user reference, and replay a recording after the collector restarts.

GOAL

Compose small runtime owners

The portable recording core has no Lustre dependency. A client owns a local delivery queue. The server coordinates PostgreSQL and S3. The optional Lustre companion composes application-owned replay modules.

NO-GOAL

Universal screen recording

The initial release reconstructs explicitly recorded application state. It does not capture pixels, operating-system dialogs, arbitrary DOM mutations or native screens.

INVARIANT

Playback does not execute application effects

The viewer uses an application-supplied pure reducer and inert view. HTTP, native calls and other effects remain in the live application. Function types alone cannot prove purity.

convention

INVARIANT

Acknowledgement follows durable commit

A Acknowledgement means PostgreSQL committed the immutable Batch. S3 failure cannot erase that accepted copy.

partial

PRINCIPLE

The application owns safe meaning

Applications select replay-safe messages, state projections, codecs, user references and Replay contract identifiers before capture. Formatters do not establish redaction.

composition

This initial system is a local development example with explicit runtime owners. Deployment authentication, tenant isolation, retention and erasure require separate policy before a hosted deployment. ADR-0001

1.2 System and ownership

The application, client queue, collector and viewer cooperate through explicit values and durable receipts.

Unit	Owns	Caller owns
Portable package	Recording cursor, reconstruction, envelope validation	Types, codecs, clock, deterministic reducer
Frontend: client modules	Bounded durable queue, serialized delivery, retries	Capture policy, endpoint, identity, storage selection

Unit	Owns	Caller owns
Server package	Atomic admission, session index, archival and retrieval	Database pool, object-store configuration, worker lifetime
Frontend: Lustre modules	Capture coordination, typed reconstruction and inert frame	Safe codecs, reducer, view and runtime lifetime
Frontend: viewer module	Default interface, playback clock and cancellable reads	Source, artifact policy, host mount and cleanup
Example applications	Basic browser adoption and advanced native/custom-adapter composition	Starting and stopping local resources

- Three packages form the distribution boundary: portable retrace; JavaScript retrace_frontend, which depends on core and Lustre; and BEAM retrace_server, which depends on core and backend libraries. Frontend delivery, capture, replay and viewer modules retain separate configuration and cleanup. Importing the package starts no resources. Tauri plugin use remains explicit. Applications can replace adapters or supply their own viewer/backend. ADR-0008
- A Session has an immutable ID, user reference, build, platform and replay contract. Multiple devices or tabs use distinct IDs. User reference is searchable metadata, not authorization.
- A recording cursor owns sequence and elapsed-time monotonicity. A Entry advances it once, including a message that changes no state. A Checkpoint starts reconstruction at its declared cursor.
- Transport format version and application replay compatibility are separate. The initial transport uses bounded JSON strings for application payloads so the collector need not import application types.
- The example uses PostgreSQL 16, a caller-configured S3-compatible service, Mist on BEAM, and a shared Lustre frontend for browser and Tauri. Nix supplies the development tools.

1.3 Recording and replay

Pure values retain application types and report invalid history without advancing a player.

BEHAVIOR

Invalid history preserves the replay cursor

Given a player at a validated checkpoint

When an entry has a gap, duplicate position, different session or contract, backwards time, or a failing application reducer

Then the operation returns the corresponding typed failure and the previous player remains usable

level: boundary area: Reconstruction

BEHAVIOR

A checkpoint can restart reconstruction

Given a compatible checkpoint and subsequent contiguous entries

When the caller reconstructs the recorded range

Then the final state and absolute sequence can be inspected and compared with a supplied checkpoint

level: boundary area: Reconstruction

- Start validates identity. Record increments sequence and returns an immutable entry. Resume validates a supplied cursor; the caller must first recover its matching outbox and state. All sequence and elapsed-time integers fit the JavaScript exact-integer range.
- A batch contains at most 128 entries and at most one MiB of encoded bytes. Its checkpoint precedes its entries; all origins match its session. A batch is nonempty. JSON decoding does not prove the application's model or event validity.
- Compatibility checks do not prove reducer equivalence or authenticate a checkpoint. The application owns those claims.

1.4 Client delivery

The client persists before uploading and removes a batch only after its matching durable receipt.

BEHAVIOR

Interrupted delivery is recoverable

Given a batch confirmed as saved locally

When the network fails or the client runtime restarts before acknowledgement

Then the batch remains available for retry with the same session and sequence

level: boundary area: Delivery

BEHAVIOR

Capture failure stays visible

Given the local queue is full or its storage write fails

When the application attempts to record another transition

Then capture reports its failure without claiming that transition was saved

level: interface area: Capture

BEHAVIOR

Typed client adapters preserve operation ownership

Given a client using application-supplied typed storage and upload operations

When an operation completes, fails or its pending upload is cancelled

Then only its first completion is observed; cancellation releases the upload and ignores late replies, while admitted storage work settles and borrowed resources remain caller-owned

level: boundary area: Delivery

- Typed adapter constructors accept cold Gleam callback operations and an explicit formatter for application-native errors. Storage receives the queue namespace and limits and supplies list, put and remove operations. Upload receives the exact encoded batch and returns the encoded receipt; the client retains receipt validation. The bridge handles synchronous completion, thrown provider errors, deadlines and late replies without adding retries or taking ownership of resources. Storage operations must settle; their cancellation callbacks are not invoked because admitted writes and deletions must finish. Default and foreign adapters remain available. No archive or build-metadata framework belongs to this module. ADR-0007
- A client runtime has one writer and one sequential upload loop. Each page instance starts a fresh session; it still drains durable batches left by earlier instances. Upload duplicates caused by overlapping runtimes are safe on the server.
- Browser persistence defaults to IndexedDB. Tauri uses an explicitly selected native store adapter. Callers can supply storage and an abortable upload transport; the runtime owns ordering and acknowledgement validation. The SDK returns explicit persistence and delivery status through the application's callback. It owns retry timers and exposes shutdown.
- Pending capture and durable storage each have independent item and encoded-byte caps. Admission reserves pending capacity synchronously; overload stops further capture visibly. Already admitted writes complete unless persistence itself fails. The upload deadline covers headers and acknowledgement body, and shutdown aborts that request. Custom storage must enforce atomic, idempotent writes and durable capacity limits; custom transports must honor cancellation. A live transition and asynchronous local capture are separate commits; an abrupt termination before save confirmation can lose that transition. Persisted batches survive a normal runtime restart; device failure, storage eviction and mobile background execution are outside this guarantee.

1.5 Ingestion and archival

The server serializes admission per session and moves accepted recording bytes to object storage without a loss window.

BEHAVIOR

Duplicates are distinguished from conflicts

Given a batch was durably accepted

When a caller retries its range

Then identical content returns the same acknowledgement and different content is refused

level: boundary area: Admission

BEHAVIOR

Session admission is ordered and atomic

Given a session's durable cursor

When a new batch is admitted

Then its checkpoint must match that cursor and its fixed session metadata must agree before the batch and index advance together

level: boundary area: Admission

BEHAVIOR

Archival failure preserves accepted data

Given a committed batch still held in the database

When object upload fails

Then the database copy remains readable and eligible for a later retry

level: boundary area: Archival

BEHAVIOR

Session archives preserve whole batches

Given an idle session with original durable batch receipts

When the host compacts it or appends a late batch

Then at most 1000 entries occupy each current generation, a session no larger than that stays in one object when archivable, and original checkpoints, page order and exact-retry identities remain unchanged

level: boundary area: Archival

BEHAVIOR

Publication has a durable recovery source

Given a persisted archive plan with its payload and fixed object key

When PUT, verification or the publishing worker fails

Then the old sources remain authoritative until verified publication commits; a replacement worker can resume the plan and a deletion fence inventories every possible generation

level: boundary area: Archival

BEHAVIOR

Archived sessions remain replayable

Given a batch was successfully archived

When the collector restarts and the viewer requests that session

Then the session remains searchable and its recording is fetched and checked before playback

level: boundary area: Retrieval

BEHAVIOR

Concurrent retrieval preserves ordered evidence

Given a caller opts into concurrent reads from an adapter safe for independent read calls

When a bounded archived range is retrieved

Then concurrency stays within the requested bounded budget, results remain in recording order, every payload is verified, and any failed object refuses the whole range

level: boundary area: Retrieval

BEHAVIOR

Read workers remain scoped to their caller

Given a concurrent range read has outstanding adapter calls

When the read completes, fails or loses its caller

Then its read workers terminate without stopping borrowed resources; a deletion fence observed before return refuses the range

level: boundary area: Retrieval

BEHAVIOR

Erasure fences delivery and archival

Given a caller-authorized session deletion

When the caller commits a fence and drives erasure

Then admission and reads refuse the session, active archival settles before the fence, objects are deleted before recording rows, and retained tombstones permit idempotent retry and late-object reconciliation

level: boundary area: Retention

- The host owns retention deadlines, authorization, backup restoration and recurring erasure. Permanent opaque tombstones prevent late delivery. Recorded object keys remain for repeated deletion after an ambiguous remote PUT; completed erasure does not establish removal from provider versions or backups.
- Batch receipts retain their original range and digest after archival. The legacy archive operation stores one batch per object; the opt-in session operation groups whole original batches greedily into segments of at most 1000 entries. A session at or below that count has one current object when archivable. Only entry count creates another segment; checkpoints and retry receipts are never rechunked. Host-configured inactivity, ordinarily 60 seconds, makes work eligible without relying on browser shutdown. ADR-0009
- Immutable archive generations move from planned to current to retired. A plan commits its key, digest and complete payload before any remote PUT. Publication verifies a GET against the exact planned bytes before clearing database batch payloads and changing pointers. A crash or uncertain PUT leaves a durable plan for retry and an inventoried key for erasure. A late batch replaces its preceding segment if the whole batch fits. Superseded generations never become current again.
- A decoded segment is limited to 32 MiB. Larger groups retain their existing sources and receive a durable deferral until the session advances; byte size does not create premature segments. At most one planned payload per session temporarily duplicates up to 32 MiB in PostgreSQL, which hosts must count in capacity policy. Decompression stops at the byte limit.
- Preparation snapshots sources under a short session lock, reads objects without holding that lock, and rechecks the original source pointers and complete segment coverage before committing a plan. Legacy reads use windows of at most sixteen one-

MiB objects and a running byte budget; existing segment objects are fetched individually. Concurrent admission beyond the snapshot range remains safe. Publication holds the session lock while uploading, verifying and replacing pointers; adapters must bound external I/O.

- Retired keys are eligible for immediate cleanup in passes of at most eight. Later DELETE reconciliation uses the host interval. Cleanup rechecks that neither batches nor an active plan reference the key; the registry retains retired keys for fencing and ambiguous late-PUT reconciliation. Old per-batch objects remain readable and are automatically eligible for idle compaction.
- Admission locks the session row. The initial checkpoint starts at sequence zero. A previously accepted batch is identified by session and starting sequence; its digest permits exact retry checks after its database payload has been removed. Different chunking is a conflict, not automatic deduplication.
- Session search is bounded and supports a user reference. Range retrieval is ordered and bounded to 128 batches. Serial reads retain their calling-process compatibility. Opt-in concurrent reads clamp their worker budget to one through sixteen and require adapters safe for independent calls. Workers belong only to the synchronous read; adapter operations must bound their own external I/O. Each page fetches a shared object once and verifies both its envelope and original batch identities. The object adapter returns compressed bytes; reconstruction verifies the exact stored content digest after bounded decompression. A missing object permits one authoritative pointer refresh only when its source changed; corrupt content never falls back. A final tombstone check refuses a read fenced while its objects were being fetched.
- The server package borrows a database pool and object-store functions. The development collector owns the pool and periodic archive worker. Storage functions have explicit read/write failures; no package starts hidden background workers.

1.6 Lustre replay composition

An optional companion reconstructs bounded ranges using application-native types and a shared inert view.

BEHAVIOR

Application adapters preserve range identity

Given an application adapter, expected build and a bounded recorded range

When the range is reconstructed from its first checkpoint

Then all envelopes, session metadata, checkpoints and transitions must agree before any snapshots are returned; failure leaves the caller free to retry a corrected range

level: boundary area: Replay composition

BEHAVIOR

Replay frames have an explicit lifetime

Given a mounted application replay frame

When the host disposes it

Then its message listener and application runtime stop responding

level: boundary area: Replay composition

BEHAVIOR

Isolated replay channels require exact authority

Given an explicitly isolated replay frame using a verified application build

When the viewer or frame receives a protocol message

Then it accepts only the expected window, exact configured origin, protocol version and per-frame nonce; the frame never infers parent authority from a message, referrer or recording

level: boundary area: Replay composition

BEHAVIOR

Capture coordination preserves application ownership

Given an initialized capture state and an application-approved transition

When the application observes that transition with its safe state and event codecs

Then the adapter records its checkpoint and event in order, reports persistence separately, and stops capture visibly on failure without executing or suppressing the application's effects

level: boundary area: Capture composition

BEHAVIOR

Viewer adapters own cancellable work

Given a viewer using default or application-supplied typed readers

When a request is replaced or the viewer is disposed

Then its pending reads are cancelled, stale replies cannot replace the current display, and disposal can be repeated safely

level: boundary area: Viewer composition

BEHAVIOR

Progressive replay exposes a usable prefix

Given a viewer configured for progressive loading of a bounded recording

When the first valid range is available and later ranges are still loading

Then the developer can inspect the loaded interval, subsequent ranges extend the same timeline without moving the playhead, and a later retrieval failure keeps the valid prefix available with an explicit retry

level: interface area: Viewer composition

BEHAVIOR

Progressive ranges preserve bounded evidence

Given a progressive reader at an accepted checkpoint

When a later page changes session identity, skips an entry, reverses time or exceeds its configured budgets

Then the page is refused without advancing the accepted cursor; a corrected retry starts from that cursor and cancellation cannot commit late data

level: boundary area: Viewer composition

- The optional capture module owns recorder coordination and persistence feedback as immutable state and Lustre effects. Applications map its messages into their update function and observe accepted transitions; delivery messages are never recorded recursively. A per-session monotonic clock and random session ID are supplied, with explicit clock injection for deterministic consumers. Capture stops visibly if observed before connection, on invalid time, or on failed persistence; the application remains usable. The caller owns the client lifetime and invokes stop when its application unmounts.
- Viewport observations expose typed CSS dimensions and safe-area insets. Applications translate these values into their own recorded message schema.
- The experimental retrace_frontend/replay module owns a typed replay-module adapter, bounded reconstruction and an inert frame. Applications supply codecs, deterministic reduction, equality, event labels, presentation projection and the complete view used live. Reconstruction retains typed application errors; the frame renders diagnostic text.
- The JavaScript viewer optionally accepts an application-resolved self-contained document in explicit opaque mode. It mounts srcdoc with allow-scripts only, without allow-same-origin, and suppresses referrers. The application assembler owns verification, a restrictive content security policy, embedded assets and a fixed trusted parent origin passed to mount_with_parent_origin. The iframe name carries a random per-load nonce. Only opaque mode uses wildcard outgoing delivery because an opaque origin cannot be named; inbound trust still requires the exact frame window, null origin and nonce. Default local frames retain same-origin URL behavior.

- Ranges may begin at a nonzero checkpoint. Snapshot and seek identifiers are absolute sequence numbers; offsets remain internal. The default viewer uses a caller-supplied range reader with an after cursor and bounded page size, validates progress and refuses beyond its retained-entry budget. The host owns storage, authorization and artifact selection.
- The optional JavaScript progressive viewer reads bounded pages and publishes windows at whole-batch boundaries, targeting five minutes, 256 entries or one MiB. A batch remains indivisible, so one long batch can span a larger time window. An exhausted page is immediately playable instead of waiting to fill the target. The selected recording retains at most its configured entry and encoded-byte budgets; no recordings survive selection or disposal. Later windows extend the validated prefix using the existing load and absolute-sequence seek protocol, without recreating the artifact frame. After first playback, checked windows from the same page are combined into one update. The viewer reads at most one page ahead while the frame reconstructs, with one read and one frame update in flight. Prefetched data counts toward the same retained budgets. The viewer reports only the acknowledged interval and pauses at its end. Failed later reads preserve the prefix and retry the uncommitted cursor.
- Applications project a fixed desktop viewport or record resize observations in their own events. An optional browser helper observes CSS dimensions and safe areas inside browsers or Tauri WebViews and returns cleanup. Presentation never enters the portable recording core.
- The basic inventory consumer uses provided browser persistence, HTTP delivery and viewer readers without application-authored FFI. The advanced membership consumer combines Tauri, native results and an optional custom local archive with its own storage, transport and typed viewer source. Both import public package APIs and retain their own application types. Exactly these two examples are maintained; the shared collector belongs to development tooling and backend fault consumers remain isolated test harnesses. ADR-0006 The companion and viewer remain experimental bounded replay, without live subscription or an unbounded cache. ADR-0003

1.7 Example application and viewer

A small membership app demonstrates the same session contract in the browser and Tauri.

BEHAVIOR

Playback preserves recorded timing

Given a complete validated recording range

When a developer plays or seeks within it

Then the playhead follows elapsed time, preserves gaps and displays the last state at or before that time; equal-time entries retain sequence order and appear together; previous and next navigation move between distinct timestamps

level: interface area: Replay

BEHAVIOR

Replay preserves the application viewport

Given a recording with application presentation state

When a developer inspects a point in time

Then the complete application view uses the recorded viewport and screen, scaled to fit without changing its responsive layout

level: interface area: Replay

BEHAVIOR

Unavailable artifacts refuse playback

Given a session whose matching replay artifact is missing or incompatible

When a developer opens it

Then the viewer explains the failure and does not substitute the current application build

level: interface area: Replay

BEHAVIOR

Replay is inert

Given a compatible stored recording

When the viewer navigates its entries

Then the application state is reconstructed without making native requests or recording new live events

level: interface area: Replay

- The live application and viewer have separate entrypoints and local endpoints. The **Replay viewer** owns search, artifact loading, timing and event navigation. The live application owns capture and effect execution. Capture diagnostics remain available as a separate live-only panel.
- A **Replay module** supplies the full view used by live and recorded states. A native platform-information request demonstrates a genuine Tauri command and records its result as an application message. The browser has its own labelled result. Native operating-system UI is represented by events, not a pixel recording.
- An application-owned **Replay artifact** checks its build and contract, validates all checkpoints and entries, and renders the same root view as the live application. The example host embeds trusted local artifacts in an iframe using a versioned, source-checked message protocol. It accepts validated checkpoint ranges and bounds retained snapshots. Time Travel is not a runtime dependency; retained attribution records the original derivation. ADR-0005
- The application records CSS viewport dimensions, safe-area insets and screen navigation in its own model and messages. The wire format remains unchanged. Replay stops at the last observed event and labels this as a loaded range, not a known session end. The viewer pauses when hidden. Native sheets, keyboard pixels, arbitrary scrolling and autonomous animation are outside this example's fidelity claim.
- The optional retrace_frontend/viewer module owns the default playback interface and typed cancellable source and artifact adapters. Its collector and artifact-directory defaults require no application-authored JavaScript. Applications can replace storage reads and artifact policy from Gleam, and customize the supplied stylesheet. These interfaces remain experimental. ADR-0005
- The local collector binds to loopback by default. Local example routes are unauthenticated and must not be treated as a hosted service configuration.

Glossary

Every term the layer declares, with the context that owns it.

Acknowledgement

owned by root

The server's confirmation that a batch is durably committed; it permits the client to remove that batch from its queue.

Batch

owned by root

An immutable checkpoint followed by a bounded contiguous range of entries, used as the unit of upload, deduplication and replay pagination. Archival groups whole batches without changing their receipts or checkpoints.

Checkpoint

owned by root

A replay-safe application model at a known session sequence and elapsed time.

Entry

owned by root

One accepted, safe application message with a sequence number and monotonic elapsed time inside a session.

Replay artifact

owned by root

A compiled replay module with its document, styles and dependencies, identified by the application build that produced a recording.

Replay contract

owned by root

The application-owned compatibility identifier for its event schema, checkpoint schema and reducer semantics.

Replay module

owned by root

Application-owned codecs, deterministic reducer and complete view used to reconstruct a recorded application state.

Replay viewer

owned by root

The session-inspection interface that selects artifacts and controls playback time independently of the live application.

Session

owned by root

One application’s ordered recording, identified independently from its user’s reference and fixed to an application build and replay contract.